Checkpoint 4 — Tabela de Símbolos e Passada de Identificação

Maria Rita Vieira Souza — 12321BCC024
Ana Clara Morais de Carvalho —

12311BCC048 Novembro de 2026

Repositório: https://github.com/mariarvsouza/GBC071-Construcao-de-Compiladores

I. Ambiente em cada ponto de um programa

O exemplo abaixo contém uma função com parâmetro, dois níveis de blocos aninhados, som-breamento de a, uso após fechamento do bloco e acesso a identificador global. A numeração é parte do

```
 1  number g = 100;
 2
 3  number soma(number x) {
 4      number a = 10;
 5      for (number i = 0; i < x; i = i + 1) {
 6          number a = 20;
 7          number b = a + i + g;
 8      }
 9      b = 1;
10      number c = a + g;
11      return c;
12  }
```

desenho e serve de referência aos vínculos nome@linha.

A linha 9 é **contextualmente inválida**: b foi declarada apenas no corpo do for e já saiu de escopo. O código ainda é sintaticamente reconhecível; o erro só aparece na identificação. A atribuição da linha 9 é propositalmente mantida para demonstrar esse caso.

I.I Ambientes após declarações e fechamentos

A notação {a@4, x@3} :: {g@1} coloca o escopo mais interno à esquerda. O escopo global é o nível 0. Para este projeto, o escopo dos parâmetros e do corpo principal da função é único. O for possui um escopo para sua inicialização e outro para o bloco do corpo.

Ponto	Ambiente ativo (interno :: externo)
Após declarar g (linha 1)	{g@1}
Abrir função e declarar x (linha 3)	{x@3} :: {g@1, soma@3}

Ponto externo)	Ambiente ativo (interno ::
Após declarar a (linha 4)	{a@4, x@3} :: {g@1, soma@3}
Após inicialização do for (linha 5)	{i@5} :: {a@4, x@3} :: {g@1, soma@3}
Ao abrir corpo do for (linha 5)	{ } :: {i@5} :: {a@4, x@3} :: {g@1, soma@3}
Após a = 20 (linha 6)	{a@6} :: {i@5} :: {a@4, x@3} :: {g@1, soma@3}
Após b (linha 7)	{b@7, a@6} :: {i@5} :: {a@4, x@3} :: {g@1, soma@3}
Fechar corpo do for (linha 8)	{i@5} :: {a@4, x@3} :: {g@1, soma@3}
Fechar escopo do for (linha 8)	{a@4, x@3} :: {g@1, soma@3}
Após uso inválido de b (linha 9)	{a@4, x@3} :: {g@1, soma@3}
Após declarar c (linha 10)	{c@10, a@4, x@3} :: {g@1, soma@3}
Fechar função (linha 12)	{g@1, soma@3}

1.2 Vínculos de usos de identificadores

Linha encontrada	Uso	Declaração
5	x na condição i < x	x@3 (parâmetro)
5 e no	i na condição incremento	i@5 (variável do laço)

7	a na expressão	a@6 (sombreamento interno)
7	i na expressão	i@5
7	g na expressão	g@1 (global)
9	b como destino de atribuição	**Não declarado neste ponto;** b@7 saiu de escopo
10	a na expressão	a@4 (externo ao laço)
10	g na expressão	g@1
11	c no retorno	c@10

O destino i do incremento do for também é uso e deve se vincular a i@5. A declaração de b não é removida da AST, mas deixa de estar disponível na tabela após o fechamento do bloco.

2. Projeto de Symbol e escolha da tabela

2.1 Campos de Symbol

Campo	Tipo	Finalidade atual / fase futura
name	String	Nome consultado por declare, lookup e mensagens de erro.
kind	Kind	Categoria VAR, PARAM ou FUNC; útil para verificações contextuais posteriores.

Campo	Tipo	Finalidade atual / fase futura
decl	Node	Referência ao nó da declaração e à linha de origem; base da identificação e análise semântica.
level	int	Profundidade do escopo de declaração.
type	Type	Reservado para inferência e checagem de tipos no CP5.
offset	int	Reservado para organização de memória / geração de código.

```
public class Symbol {
```

```
    public final String name;
    public final Kind kind;
    public final Node decl;
    public final int level;
    public Type type;
    public int offset;

    public Symbol(String name, Kind kind, Node decl, int level) {
        this.name = name;
        this.kind = kind;
        this.decl = decl;
        this.level = level;
        this.type = null;
        this.offset = -1;
    }
}
```

O arquivo Type.java da implementação atual é apenas um marcador provisório: a checagem dos tipos **não** faz parte deste checkpoint. A categoria Kind não substitui a análise de tipos nem implica, por si só, que o Visitor atual já reporte usos de categoria incorreta.

2.2 Implementação da API SymbolTable

Adotou-se a **pilha de mapas** (StackSymbolTable, implementada com Deque<Map<String, Symbol>>), em vez de escopos encadeados ou tabela única com registro. Ela representa diretamente cada escopo ativo, permite sombreamento e restaura a declaração externa ao desempilhar um mapa. O Visitor usa a interface SymbolTable, e não depende da classe concreta, salvo na criação da tabela.

```
interface SymbolTable {
    void openScope();
    void closeScope();
    boolean declare(Symbol s);
    Symbol lookupLocal(String name);
    Symbol lookup(String name);
    int level();
}
```

Contrato observado na implementação: a tabela inicia com o global aberto (level() ==
0); openScope() empilha um mapa; closeScope() desempilha o escopo atual. Tentar fechar o global gera IllegalStateException com mensagem explicativa. declare retorna false para nome já presente no escopo atual e conserva a primeira declaração. lookupLocal procura apenas no topo, enquanto lookup busca de dentro para fora; ambos retornam null se não houver símbolo.

3. As cinco decisões de projeto

3.1 Sombreamento e redeclaração

Escolha: permitir sombreamento de um nome externo por outro em escopo interno e proibir redeclaração no mesmo escopo.

Alternativa descartada: impedir todo sombreamento ou permitir duplicatas locais.

Motivo: é possível reutilizar nomes em blocos distintos sem tornar ambígua a busca local; o mapa mais interno tem prioridade.

```
number a = 1;
number teste(number b) {
    number a = 2;
    return a + b;
}
number c = 3;
number c = 4;
```

O a interno é válido; a segunda declaração de c deve reportar redeclaração e preservar c da primeira linha correspondente.

3.2 Declaração antes do uso

Escolha: nomes de variáveis precisam estar declarados antes de serem usados.

Alterna-tiva descartada: pré-passada de declarações de variáveis.

Motivo: uma única visita na ordem textual simplifica o controle dos ambientes.

```
x = 10;
number x = 5;
```

O primeiro x não se resolve; o segundo comando declara x apenas a partir de sua posição.

3.3 Recursão direta e mútua

Escolha: permitir recursão direta e não permitir referência antecipada em funções mutua-mente recursivas.

Alternativa descartada: registrar previamente todas as funções antes de visitar os corpos.

Motivo: a implementação identifica nomes em uma única travessia; a própria função é declarada no escopo externo antes do processamento dos parâmetros e corpo.

Exemplo com **recursão mútua rejeitada**:

```
number f() { g(); return 1; }
number g() { f(); return 1; }
```

A chamada a g dentro de f deve reportar g não declarado. A chamada posterior a f, dentro de g, já consegue encontrar a primeira função.

Exemplo com **recursão direta aceita na identificação**:

```
number contagem(number n) {
    if (n > 0) {
        return contagem(n - 1);
    }
    return 0;
}
```

A identificação verifica a existência dos nomes, **não** a compatibilidade de parâmetros ou tipos retornados.

3.4 Parâmetros e corpo principal

Escolha: parâmetros e variáveis locais do corpo principal da função pertencem ao mesmo es-copo.

Alternativa descartada: abrir um escopo adicional para o bloco principal da função.

Motivo: evita que uma variável no corpo principal oculte um parâmetro imediatamente, mantendo uma região semântica única. Blocos aninhados continuam abrindo escopos inde-pendentes.

```
number soma(number a) {
    number a = 10;
    return a + 1;
}
```

O segundo a deve produzir erro de redeclaração no escopo do parâmetro.

3.5 Espaço de nomes

Escolha: variáveis e funções compartilham um único espaço de nomes.

Alternativa descartada: mapas separados por categoria.

Motivo: facilita as operações de declaração e consulta, impedindo duplicatas locais entre categorias.

```
char x = 'a';
number x(number y) {
    return y;
}
```

A função `x` conflita com a variável `x` no mesmo escopo global.

4. Passada de identificação: integração com a AST

A nova classe VisitorIdentificacao implementa Visitor<Void> e percorre apenas ár-vores produzidas por Parser.ParseResult quando isSucesso() é verdadeiro. A visita acu-mula erros sem interromper o processamento.

- **Programa:** visita os comandos sequencialmente, usando o escopo global existente.

- **Função:** registra o nome no escopo externo antes de abrir o escopo de parâmetros e corpo; isso viabiliza recursão direta.
- **Bloco comum:** abre um escopo, percorre os comandos na ordem e fecha o escopo; usa finally para garantir o fechamento.
- **Laço for:** abre um escopo para a inicialização/controle e visita seu bloco, que abre outro escopo.
- **Declarações:** criam Symbol(name, kind, decl, level) e utilizam declare; se falhar, registram a declaração anterior sem sobrescrevê-la.
- **Usos:** consultam lookup. Em IdExprNode, o campo decl recebe o Symbol encontrado ou permanece null quando não declarado.
- **Usos armazenados como texto no CP3:** AssignNode.varID e CallFuncExprN-ode.func são resolvidos pelo Visitor e associados aos seus nós através de um mapa externo (getResolucao(Node)). Assim, não se altera a estrutura dessas duas classes.

Formato determinístico dos erros:

```
linha 2: 'a' já declarado neste escopo (linha 1)
linha 3: 'b' não declarado
```

A implementação atual faz *resolução de nomes*. O campo kind é registrado, mas a validação de se uma chamada é realmente uma função e se uma atribuição tem destino permitido deverá ser incorporada na fase semântica correspondente; não se deve afirmar que isso já é verificado.

5. Suíte de testes e resultados esperados

A execução foi realizada no projeto do grupo, conforme informado após rodar Symbol-TableTest, IdentificacaoTest e Checkpoint4Suite. O pacote de testes do CP4 contém test/programas/, test/esperado/ e test/esperado_cp3/.

5.1 Tabela isolada

O SymbolTableTest.java exercita declaração/busca, redeclaração sem substituição, som-breamento e restauração após closeScope, escopo local, nome inexistente, profundidade (level) e comportamento documentado ao tentar fechar o global.

5.2 Programas de identificação

Programa em test/programas/	Verificação principal
sombreamento_valido.txt	Usos resolvem declaração interna ou externa conforme o escopo.
for_valido.txt	Inicialização, condição, incremento e corpo do for.
recursao_direta.txt	A função encontra seu próprio nome e seu parâmetro.
redeclaracao.txt	Nome repetido no mesmo bloco gera erro; a primeira

	declaração fica.
nome_nao_declarado.txt	Uso de identificador inexistente gera erro.
declarar_antes.txt	Referência anterior à declaração é rejeitada.
recursao_mutua.txt	Chamada antecipada à segunda função é rejeitada.
parametro_corpo.txt	Parâmetro e variável local não podem repetir o nome no mesmo escopo.
espaco_nomes.txt	Variável e função com mesmo nome conflitam.
for_escopos.txt	Sombreamento interno e uso de nome após fechamento do bloco.
multiplos_erros.txt	Diversos erros são acumulados em ordem, sem parar no primeiro.

O teste multiplos_erros.txt, por exemplo, possui a seguinte **saída esperada**:

```
linha 2: 'a' já declarado neste escopo (linha 1)
linha 3: 'b' não declarado
linha 3: 'c' não declarado
linha 4: 'e' não declarado
```

O relatório de vínculos usa linha N: 'nome' -> KIND@M, sendo M a linha da declaração; apresenta os erros primeiro e depois os vínculos na ordem de visita à AST. Os arquivos de referência estão em test/esperado/ e devem ser conferidos pelo grupo com o desenho do ambiente, independentemente da execução.

5.3 Regressão do CP3

A suíte verifica a continuidade da impressão da AST para os programas válidos correspon-dentes aos Testes 1, 2 e 4, o comportamento do reimpressor de código (ida e volta) e a rejeição de entradas sintaticamente inválidas. O antigo Teste 3 do CP3 é inválido e, por isso, **não** deve produzir AST válida comparável a um arquivo de árvore esperada.

Resultado: os testes isolados da tabela, os testes básicos de iden-tificação e a suíte integrada foram executados com sucesso. Uma entrega reprodutível deve manter os arquivos test/esperado/ e test/programas/ junto ao código; prints ou log de console podem ser anexados como evidência adicional.